Project 3 - Offline Goal-Conditioned Reinforcement Learning - Phase 1

Code Repository Link: Offline GCRL - Phase 1.

We use the official repositories and benchmark implementations for these offline goal-conditioned RL methods, with experiments run through a consistent training and evaluation pipeline. This helps ensure reproducibility and allows fair comparison across algorithms and environments, including both AntMaze and visual goal-reaching tasks.

Algorithms

We evaluate five offline goal-conditioned reinforcement learning methods: **GCIQL**, **HIQL**, **QRL**, **TMD**, and **MQE**. All are tested on **OGBench** environments, where the goal is to learn from offline data and reach target states over long horizons.

GCIQL (Goal-Conditioned Implicit Q-Learning) adapts implicit Q-learning to goal-conditioned tasks. It learns value functions directly from offline data and derives a goal-conditioned policy without explicit maximization over out-of-distribution actions.

HIQL (Hierarchical Implicit Q-Learning) extends GCIQL with a hierarchical policy structure. A high-level policy selects intermediate subgoals, while a low-level policy executes actions to reach them. This improves performance on long-horizon tasks by enabling temporal abstraction and stitching through waypoints.

QRL (Quasimetric Reinforcement Learning) imposes a quasimetric structure on goal-reaching value functions. By modeling temporal distance-to-goal with geometric constraints, it aims to improve value estimation and generalization.

TMD learns a goal-conditioned temporal distance representation that measures progress toward a target goal. This structured notion of distance helps guide policy learning in long-horizon tasks.

MQE (Multistep Quasimetric Estimation) combines quasimetric structure with multistep value propagation. It is designed to better capture long-horizon dependencies and improve stitching across trajectory segments.

Tasks

We evaluate the algorithms on seven tasks from OGBench, a benchmark suite for offline goal-conditioned reinforcement learning. The tasks span two families: state-based manipulation environments and AntMaze navigation environments in which an 8-DoF Ant robot traverses large mazes.

1. **Scene Play**: A manipulation task where a robotic arm rearranges objects to reach a goal configuration from a diverse play dataset.
2. **Scene Noisy**: A variant of Scene Play with corrupted observations and transitions, testing robustness to imperfect offline data.
3. **Cube Double Play**: A manipulation task requiring the agent to move two cubes to target positions from a play dataset.
4. **PointMaze Giant Navigate**: A navigation task in a large maze using a navigation dataset.
5. **AntMaze Medium Stitch**: An Ant navigation task on a medium maze with a stitch dataset. Stitch datasets are deliberately structured so that no single trajectory spans the full start-to-goal path. The agent must combine segments from multiple suboptimal trajectories, which requires multi-step dynamic programming and cannot be solved by behavioral cloning or one-step methods.
6. **AntMaze Large Stitch**: The same stitch task on a larger maze, increasing horizon length and reducing per-trajectory state-space coverage.
7. **AntMaze Giant Stitch**: The largest and most challenging AntMaze stitch task.

Experimental Setup

Our study consists of 137 total runs across 9 OGBench environments, covering both state-based (7 environments) and pixel-based tasks (2 environments). For pixel environments, we used 9 runs per environment (1 seed for MQE and 4 seeds each for HIQL and TMD). For state environments, we used 17 runs per environment (1 MQE seed and 8 seeds for the other methods), following the original evaluation protocol.

We used consistent training settings across all experiments. The batch size was 256, and the discount factor was $\gamma = 0.995$. MQE used a waypoint discount $\lambda = 0.95$ and sampling probabilities $p = 0.2$ (state) and 0.1 (pixel). Policy and encoder networks were implemented as 3-layer MLPs with hidden size 512, while the Metric Residual Network used 8 components.

All experiments were run on NVIDIA V100 16GB GPUs with a learning rate of 3e-4. Behavior cloning coefficients α were environment-dependent: 0.03 for AntMaze stitch tasks, 1.0 for scene and cube tasks, and 3.0 for visual variants.

Results & Analysis

The quantitative results of our reproduction study are summarized in Table 1. We additionally show qualitative results in Fig. 2, comparing goal configurations with the final states reached by successful policies. Overall, the results highlight that different algorithmic design choices lead to strengths in different environment types.

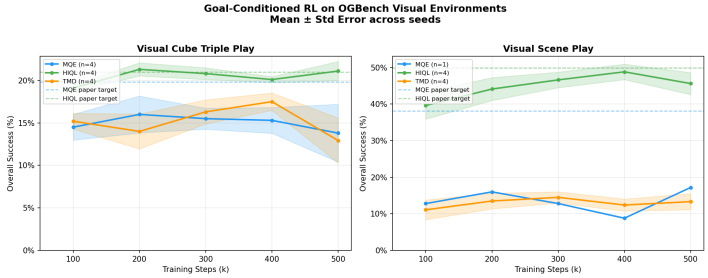

Figure 1: *Visual scene play and visual cube triple cube play*

Maze-based Tasks. In long-horizon navigation and stitching tasks, HIQL consistently demonstrates the strongest and most reliable performance. It achieves the best results in `antmaze_medium_stitch` ($94.6 \pm 1.3\%$) and `antmaze_large_stitch` ($71.4 \pm 5.9\%$), suggesting that hierarchical subgoal learning is particularly effective for trajectory stitching problems. QRL performs well on the pure navigation task `pointmaze_giant_navigate` ($65.7 \pm 8.8\%$) but fails to generalize to more complex stitching environments. MQE shows moderate performance on navigation but fails to scale to larger stitch environments in our experiments.

Manipulation Tasks. In scene and cube manipulation tasks, GCIQL and HIQL achieve the strongest results. GCIQL performs best on `scene_play` ($52.3 \pm 3.1\%$) and `cube_double_play` ($27.9 \pm 5.7\%$), indicating that direct value learning from offline data remains a strong baseline for structured manipulation problems. HIQL remains competitive and achieves the best result in the noisy setting `scene_noisy` ($21.4 \pm 2.6\%$), suggesting improved robustness from hierarchical decomposition. MQE performs poorly in these environments, indicating limitations of distance-based objectives for multi-object manipulation.

Visual Tasks. In pixel-based environments, HIQL significantly outperforms other methods, achieving $49.8 \pm 2.2\%$ on `visual_scene_play` and $20.8 \pm 2.4\%$ on `visual_cube_triple_play`. This suggests that hierarchical subgoal representations transfer more effectively to high-dimensional visual observations than global distance-based approaches such as MQE or TMD.

Summary. Overall, HIQL shows the most consistent performance across environment types, particularly in long-horizon and visual tasks. GCIQL remains a strong baseline for manipulation tasks, while QRL shows strengths in navigation but limited generalization. MQE and TMD show mixed performance in our reproduction, suggesting sensitivity to environment structure or implementation details.

Discussion and Future Work

Although we ran MQE on all environments using the reported hyperparameters, we were unable to fully reproduce the reference paper's values, suggesting sensitivity to implementation details or task-specific tuning. In contrast, HIQL was the most consistent method across long-horizon and visual tasks in our runs. Our next step is therefore to improve HIQL by exploring: (1) multistep value propagation, (2) structured distance-based critics such as quasimetric or temporal-distance learning, and (3) stronger latent subgoal representations or contrastive auxiliary objectives. These directions aim to preserve HIQL's strong hierarchical structure while improving its long-horizon value estimation.

Table 1: OGBench Evaluation

	Methods				
Dataset	**MQE**	**TMD**	**QRL**	**GCIQL**	**HIQL**
pointmaze_giant_navigate	60.0(±0.0)	–	65.7(±8.8)	–	40.1(±17.4)
antmaze_medium_stitch	36.8(±0.0)	93.2(±0.0)	–	–	94.6(±1.3)
antmaze_large_stitch	0.0(±0.0)	62.4(±0.0)	–	–	71.4(±5.9)
antmaze_giant_stitch	0.0(±0.0)	1.2(±0.0)	–	–	2.7(±2.3)
scene_play	1.6(±0.0)	–	–	52.3(±3.1)	43.7(±4.3)
scene_noisy	7.2(±0.0)	–	–	19.7(±4.7)	21.4(±2.6)
cube_double_play	0.0(±0.0)	–	–	27.9(±5.7)	11.6(±4.2)
visual_scene_play	17.2(±0.0)	12.4(±3.3)	–	–	49.8(±2.2)
visual_cube_triple_play	13.5(±3.3)	14.7(±4.7)	–	–	20.8(±2.4)

Results are reported as mean ± standard deviation.

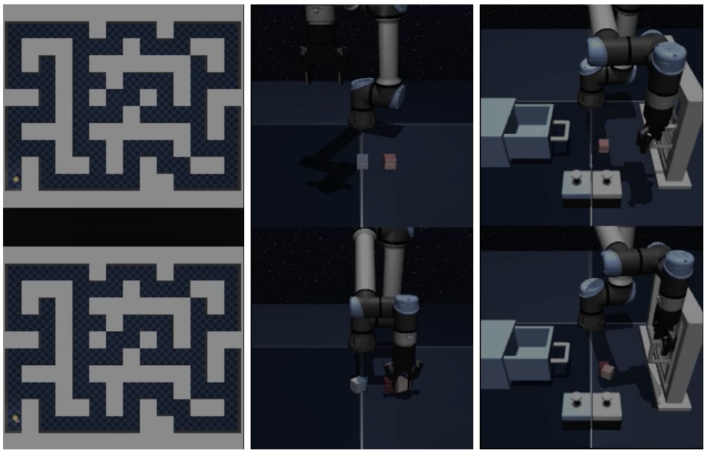

Figure 2: *Qualitative comparison between goal states (top row) and final states from successful executions (bottom row). From left to right: PointMaze navigation, Cube Double Play, and Scene Play. The learned goal-conditioned policies are able to reach or closely approximate the desired goal configurations across both navigation and manipulation tasks.*

Project 7 - Multi Agent RL - Phase 1

Code Repository Link: Multi Agent RL - Phase 1.

We use **BenchMARL** as the experimental framework for this phase. BenchMARL is a multi-agent reinforcement learning benchmark library built on top of TorchRL, with a strong emphasis on modular configuration, reproducibility, and fair comparison across algorithms and environments. BenchMARL organizes experiments through reusable YAML configurations and a unified training and evaluation pipeline, enabling consistent and reproducible comparisons across settings.

Algorithms

We study two PPO-based multi-agent baselines: **IPPO** and **MAPPO**. Both are actor–critic methods derived from PPO, but they differ in the information available to the critic during training.

IPPO (Independent PPO) models each agent as an independent learner with a decentralized policy and a critic based on local information. This makes IPPO conceptually simple and scalable, but it may be less effective in environments requiring strong coordination.

MAPPO (Multi-Agent PPO) follows the centralized-training, decentralized-execution paradigm. During execution, each agent acts using only its own observation, but during training the critic can access richer global information about the environment and other agents. This additional information can improve value estimation and coordination, making MAPPO a stronger baseline in settings where joint behavior is important.

This contrast allows us to directly evaluate the impact of centralized value estimation on multi-agent coordination.

Tasks

We evaluate the algorithms on four tasks spanning two environment families: **VMAS** and **PettingZoo**.

1. VMAS Navigation: The goal is for agents to move efficiently to their targets without interfering with one another. Formally, this is a cooperative continuous-control task with 3 agents evolving in a shared 2D environment. In our setup, collisions are enabled and each episode has a fixed horizon of 100 steps. The reward reflects coordinated progress toward task completion, while the interaction dynamics require agents to account for the presence of others in a shared space. Performance depends on both individual control and multi-agent coordination.

2. VMAS Balance: The goal is for the team to jointly stabilize and transport a shared object. This task involves 4 agents and a horizon of 100 steps. Unlike purely independent navigation, the environment couples the agents through a common physical objective, so the effect of one agent's action depends strongly on the actions of the others. The reward therefore reflects collective control quality, making this task a useful setting for evaluating whether centralized value estimation improves coordinated behavior.

3. PettingZoo Multiwalker: The goal is to carry the package forward without the walkers collapsing. This is a cooperative locomotion task with 3 walkers and a maximum episode length of 500 steps. The reward includes forward-progress incentives together with penalties for instability: in our configuration, a fall incurs a penalty of `-10`, and if a fall triggers episode termination, an additional termination penalty of `-100` is applied. Parameters like terminate_on_fall & remove_on_fall are set to True, making failure events significant both in return and in episode length. Consequently, strong performance requires both sustained forward motion and stable coordinated locomotion.

4. PettingZoo Simple World Comm: This is a cooperative multi-agent environment with asymmetric observations and explicit communication. A designated leader agent has access to richer information, while the other agents act from limited local observations and must coordinate using the shared signal. Performance depends on how effectively the team uses communicated information to reach relevant targets while avoiding distractors.

Overall, these tasks span coordination styles ranging from loose navigation to tight physical control and asymmetric cooperation, allowing us to rigorously evaluate multi-agent learning paradigms across diverse interaction patterns.

Experimental Setup

Our phase-1 study consists of **24 experiments**: 2 algorithms (**IPPO, MAPPO**) $\times$ 4 tasks (**VMAS Balance, VMAS Navigation, PettingZoo Multiwalker, PettingZoo Simple World Comm**) $\times$ 3 seeds (**0, 1, 2**). All runs were launched through the BenchMARL framework. To keep the comparison fair and easy to interpret, each run used a training budget of **2,000,000 frames**, learning rate **3e-4**, entropy coefficient **0.01**, and the same underlying hardware. For these on-policy runs, BenchMARL collected **6000** frames per training iteration, resulting in roughly **334** total iterations per run. The evaluation was performed every **120,000** frames over **10 evaluation episodes**. Both policy and critic used MLP-based models with two hidden layers of size **256**. Overall, this setup provides a compact and diverse reproducibility study of PPO-based multi-agent baselines, covering cooperative tasks with varying coordination, control, and communication demands. We evaluate reproducibility using both quantitative and qualitative evidence.

Results & Analysis

Quantitatively, we summarize each method by its best evaluation reward, final evaluation reward, and the percentage

Task	Algo	Best	Final	Drop%
Navigation	MAPPO	1.17±0.04	0.97±0.08	16.8
Navigation	IPPO	1.14±0.02	0.87±0.19	23.7
Balance	MAPPO	54.06±2.38	45.37±2.78	16.1
Balance	IPPO	52.08±2.05	36.65±9.13	25.8
MWalker	MAPPO	5.98±2.59	3.02±1.14	49.4
MWalker	IPPO	9.74±0.92	-4.02±15.6	141.3
SWComm	MAPPO	39.92±0.66	30.16±9.41	24.5
SWComm	IPPO	38.38±3.54	32.08±3.68	16.4

Table 2: *Aggregate evaluation summary across seeds. Best = peak mean reward, Final = last evaluation reward, Drop% = %age drop from peak to final.*

Figure 3: *Reward Curves for the 4 Environments*

drop from peak to final performance, aggregated across three seeds. Qualitatively, we inspect evaluation rollouts through representative early, mid, and late snapshots.

Navigation. Both algorithms learn effective navigation policies, but MAPPO is slightly stronger and more stable (Table 2, Fig. 3). MAPPO achieves better peak and final performance with a smaller drop, indicating more reliable training. Qualitatively, rollout snapshots show improved spatial separation and more direct trajectories over time, with reduced interference between agents in the late stage (Fig. 4). Overall, centralized value estimation provides a modest but consistent advantage in both stability and final performance.

Balance. Both algorithms achieve strong performance, but MAPPO is clearly more stable and attains better final results (Table 2, Fig. 3). While peak performance is comparable, MAPPO retains performance more effectively and exhibits lower variance across seeds. Qualitatively, successful runs show agents coordinating to stabilize and transport the shared object, whereas less stable policies fail to maintain consistent control over longer horizons (Fig. 4). Overall, Balance highlights a clear advantage of centralized value estimation in tightly coupled multi-agent control.

Multiwalker. Multiwalker shows the clearest trade-off between peak performance and stability (Table 2, Fig. 3). IPPO

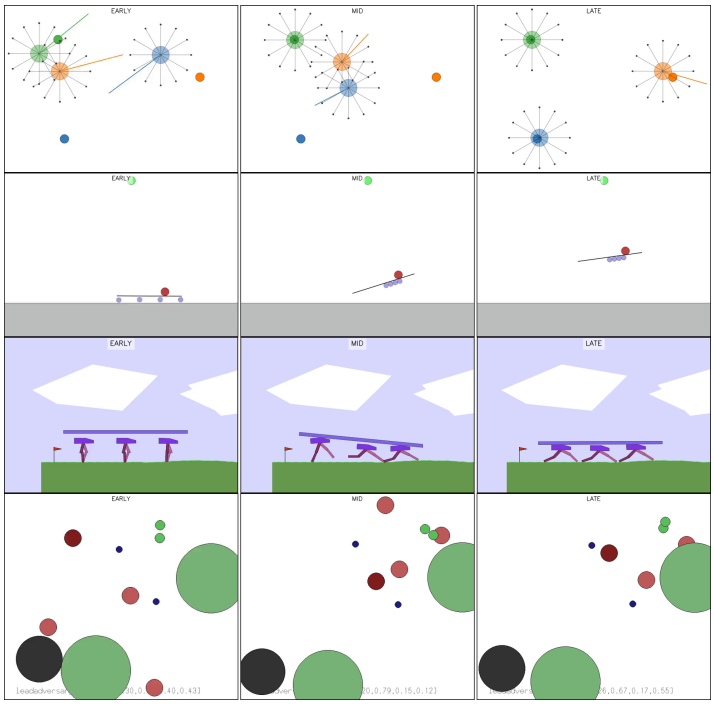

Figure 4: *Representative Rollouts with early, mid, and late stage snapshots. (From Top: Navigation, Balance, Multiwalker, Simple World Comm)*

achieves higher peak rewards but suffers from severe degradation and high variance, often collapsing by the end of training. In contrast, MAPPO maintains lower but significantly more stable and positive final performance. Qualitatively, stable runs sustain coordinated locomotion and consistent package transport, whereas unstable ones collapse over longer horizons (Fig. 4). Overall, MAPPO provides more reliable and reproducible performance in this long-horizon coordination task.

Simple World Comm. In Simple World Comm, both algorithms learn effective policies and achieve comparable peak evaluation rewards (Table 2, Fig. 3). MAPPO attains a slightly higher average best reward, but IPPO retains performance better by the end of training and exhibits lower variance across seeds. Qualitatively, rollout snapshots suggest increasingly coordinated behavior, with agents showing more purposeful movement and organization in later stages (Fig. 4). Overall, these results indicate that although centralized value estimation can discover strong solutions, IPPO provides more stable and reproducible final performance in this communication-dependent cooperative task.

Reproducibility Takeaways. Across these tasks, MAPPO generally provides more stable training and stronger final performance than IPPO. However, Simple World Comm reveals an important nuance: although MAPPO achieves comparable peak performance, IPPO is more stable across seeds and retains performance better by the end of training. Overall, these results suggest that centralized value estimation is often helpful in cooperative settings, but its benefits still depend on the structure of coordination required by the task.